

William Jackson

Senior Software Engineer- Python AI/ML

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjackson.pro@outlook.com | [linkedin.com/in/william-jackson-75b9343b6](https://www.linkedin.com/in/william-jackson-75b9343b6)

PROFESSIONAL SUMMARY

Python AI/ML Engineer with ~10 years delivering production platforms across **machine learning**, **data-intensive SaaS**, and **API-first** systems in operational analytics, workflow automation, and decision-support environments. Strong hands-on expertise building **Python 3.8–3.11** services, training and inference workflows, and cloud-native delivery patterns using **FastAPI 0.9x–0.11x**, **PyTorch 1.10–2.x**, **TensorFlow 2.x**, and event-driven orchestration on **AWS**. Known for improving reliability through **structured logging**, **OpenTelemetry**, **Datadog**, safe retry patterns, and data-quality controls that reduced noisy failures, stabilized releases, and kept model behavior explainable under real production traffic.

SKILLS

- **Languages & ML** — **Python 3.8–3.11**, **Java 8–17**, **SQL**, **scikit-learn 1.x**, **PyTorch 1.10–2.x**, **TensorFlow 2.x**, **pandas**, **NumPy**, feature engineering, model evaluation, offline/online consistency, inference optimization, batch and streaming ML workflows
- **Backend & APIs** — **REST**, **OpenAPI/Swagger**, microservices, event-driven architecture, idempotency, retries, circuit breakers, backpressure, rate limiting, worker patterns, service-to-service authentication
- **Data, Messaging & Search** — **PostgreSQL**, **MySQL**, **SQL Server**, **Redis 5–7**, **MongoDB**, **DynamoDB**, **AWS SQS/SNS**, **Kafka 2.x–3.x**, **RabbitMQ 3.x**, **Elasticsearch 7.x**, **OpenSearch 1.x–2.x**, **Redshift**, **BigQuery**
- **Cloud & DevOps** — **AWS (EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM)**, **Azure (AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights)**, **GCP (GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery)**, **Docker**, **Kubernetes 1.2x–1.2x+**, **Helm 3.x**, **Argo CD**, **GitOps**, **Terraform 1.x**, **CloudFormation**, **GitHub Actions**, **GitLab CI**, **Jenkins**, **Azure DevOps**, **CircleCI**
- **Observability, Quality & Security** — **OpenTelemetry 1.x**, **Datadog**, **Prometheus**, **Grafana**, **ELK/EFK**, **CloudWatch**, structured logging, **Jest**, **Mocha**, **Chai**, API contract tests, **OAuth2**, **OpenID Connect**, **JWT**, **Okta**, **Auth0**, **AWS Cognito**, **RBAC/ABAC**, audit logging

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viario Networks Inc. - Delaware, United States

- Designed and operated **Python 3.10–3.11** machine learning services that supported high-volume operational decisioning, translating ambiguous modeling requests into production-ready pipelines with measurable latency, accuracy, and recoverability targets.
- Built supervised learning workflows using **scikit-learn 1.x** and **PyTorch 1.10–2.x**, where stronger feature validation and leakage controls improved offline-to-production consistency by **28%** and reduced unstable retraining outcomes across releases.
- Engineered reproducible dataset generation on **PostgreSQL** and **DynamoDB**, solving frequent label drift and source mismatch issues by enforcing extraction contracts, deterministic snapshots, and audit-friendly backfill logic.
- Implemented evaluation harnesses and experiment-tracking patterns that exposed threshold regressions early, preventing model promotions that previously caused noisy false positives and protecting downstream customer-facing workflows.
- Shipped low-latency inference APIs through **FastAPI 0.9x–0.11x**, **API Gateway**, **Lambda**, and containerized workloads on **EKS/ECS**, selecting execution patterns based on burst traffic, warm-start behavior, and runtime cost.
- Resolved recurring timeout and duplicate-scoring incidents by introducing **idempotency keys**, retry boundaries, and replay-safe consumer design, which reduced production scoring failures by **40%** during peak processing windows.
- Implemented scheduled retraining and batch scoring with **EventBridge** and **Step Functions**, removing fragile manual coordination and enabling predictable model refresh cycles with rollback-aware promotion controls.

- Built streaming feature ingestion with **Kafka 2.x–3.x** and **AWS SQS/SNS**, solving message reprocessing and stale-event bugs through ordering rules, dead-letter handling, and validation checkpoints before feature publication.
- Introduced **Redis/ElastiCache** for hot feature retrieval and prediction caching, cutting repeated database pressure by **35%** while preserving correctness through controlled TTL policies and explicit invalidation paths.
- Enabled large-scale model debugging by indexing prediction traces and slice-analysis artifacts into **OpenSearch 2.x** and **Elasticsearch 7.x**, making silent performance degradation visible long before it escalated into customer issues.
- Instrumented training and inference paths with **OpenTelemetry 1.x**, **Datadog**, **Prometheus**, **Grafana**, and **CloudWatch**, turning vague incident reports into traceable failure chains that reduced MTTR by **45%**.
- Secured sensitive model endpoints with **OAuth2**, **OpenID Connect**, **JWT**, **AWS Cognito**, **RBAC/ABAC**, and audit logging, ensuring production ML services met stricter access-control and traceability expectations.

Software Engineer

October 2018 to January 2021

Avantia, Inc. - Ohio, United States

- Built production-oriented **Python 3.7–3.9** training pipelines that converted noisy behavioral and transactional data into curated datasets, helping the team standardize model inputs across multiple internal analytics workflows.
- Developed and tuned models with **scikit-learn 0.24–1.x** and **PyTorch 1.x**, introducing repeatable evaluation scripts and calibration logic that improved decision-threshold stability by **22%** across release cycles.
- Created feature storage and dataset snapshot strategies on **PostgreSQL** and **MySQL**, solving reproducibility issues that had previously made historical experiment comparisons unreliable and difficult to audit.
- Used **MongoDB** for semi-structured event payloads and evolving customer attributes, then normalized those signals into durable training features that kept inference behavior aligned with dataset assumptions.
- Delivered online scoring services in **Docker** and **Kubernetes**, addressing memory spikes and uneven request concurrency with worker isolation, queue backpressure, and safer runtime configuration patterns.
- Built batch scoring workers on **AWS SQS/SNS** and asynchronous job pipelines, fixing duplicate-processing bugs and delayed result delivery by tightening retry behavior and separating transient from terminal failures.
- Published scoring and training events into **Kafka** and **RabbitMQ**, enabling replay, recovery, and downstream analytics while preventing consumer drift through schema checks and version-aware event handling.
- Added **Redis** caching for frequently requested predictions and reusable reference features, which improved throughput by **30%** during high-demand periods without introducing stale-response risks into core workflows.
- Shipped telemetry into **Redshift** and **BigQuery** for drift analysis, segment performance review, and release comparison, making production model behavior easier to explain to technical and business stakeholders.
- Introduced better monitoring through **CloudWatch**, **Prometheus**, **Grafana**, and **Datadog**, which exposed pipeline lag, scoring anomalies, and container-health issues before they could trigger larger incidents.
- Secured multi-tenant prediction services with **OAuth2**, **JWT**, and role-based access controls, while strengthening CI/CD in **GitLab CI** and **Jenkins** to prevent unvalidated model changes from reaching production.

Software Developer

March 2016 to September 2018

ArnAmy, Inc. - Florida, United States

- Built **Python 3.6–3.8** data-processing pipelines that transformed raw operational records into ML-ready datasets, giving the team a dependable path from inconsistent source feeds to trainable features.
- Implemented labeling rules and dataset validation checks that caught malformed records, outlier-heavy segments, and missing joins early, reducing noisy training inputs by **25%** across recurring batch cycles.
- Developed baseline and iterative models with **scikit-learn**, establishing repeatable evaluation practices that replaced ad hoc accuracy checks with clearer precision, recall, and threshold tradeoff reporting.
- Designed feature extraction workflows that aligned training logic with runtime computation, solving early offline/online drift problems that had caused model behavior to change unexpectedly after deployment.
- Stored curated datasets and feature tables in **PostgreSQL** and **MySQL**, improving query performance through indexing, query tuning, and extraction cleanup for larger historical training windows.

- Introduced **Redis** caching for reused reference signals during feature generation, lowering unnecessary recomputation and helping recurring training jobs complete faster under limited infrastructure capacity.
- Used **MongoDB** to manage evolving signal structures that did not fit rigid schemas well, while still enforcing normalization steps that kept downstream feature generation stable and predictable.
- Added debugging visibility by indexing entities, outcomes, and inspection views into **Elasticsearch 5.x–6.x**, allowing quick slice analysis when model outputs looked inconsistent or unexpectedly biased.
- Scheduled batch runs with retry logic and operational notifications, resolving fragile overnight failures by separating data-quality exceptions from infrastructure interruptions and improving recovery speed.
- Containerized ML utilities with **Docker**, reducing environment drift between local development, QA, and production execution, and making dependency-related failures far less frequent during releases.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Built backend services that supported early dataset preparation, internal reporting, and analytics-oriented workflows, with strong attention to correctness, data traceability, and dependable handoff between systems.
- Implemented stable **REST** endpoints with structured validation and predictable error handling, reducing confusion for internal consumers and making service behavior easier to test and troubleshoot.
- Designed disciplined data-access patterns on **SQL Server**, solving integrity and duplication issues in source tables that were frequently reused for reporting and downstream analytical workloads.
- Developed ETL-style jobs and reconciliation scripts that replaced manual cleanup steps, which improved dataset consistency and reduced repeated operational corrections across reporting cycles.
- Added unit-test coverage with **Mocha** and **Chai** around core processing logic, helping the team catch rule regressions early and strengthening confidence in each deployment.
- Supported asynchronous job execution with **RabbitMQ**, addressing long-running task bottlenecks by improving retry, backoff, and queue handling for background processing workloads.
- Introduced **Redis** caching for read-heavy lookup paths, which reduced avoidable database pressure and improved response times for internal applications during heavier reporting activity.
- Implemented early **Elasticsearch** indexing to support faster filtering and investigative search, eliminating several slow query patterns that had depended on repeated full table scans.
- Improved build and CI hygiene by standardizing package, test, and deployment steps, reducing environment-specific failures and creating a more repeatable engineering workflow for the team.

EDUCATION

Bachelor's degree in Computer Science
Stanford University Department of Computer Science

2010 to 2014

- Completed a rigorous CS curriculum emphasizing **systems**, **databases**, and **software engineering**, with hands-on project work that strengthened practical delivery and debugging fundamentals.